

AI ASSISTED CODING

Experiment 7: Error Debugging with AI: Systematic approaches to finding and fixing bugs

NAME : G.Vijaya Simha Reddy

ROLL NO : 2503A52671

BATCH : 23

Task Description #1 (Syntax Errors – Missing Parentheses in Print Statement):

Task: Provide a Python snippet with a missing parenthesis in a print statement (e.g., `print "Hello"`). Use AI to detect and fix the syntax error.

Bug: Missing parentheses in print statement

```
def greet():  
    print "Hello, AI Debugging Lab!"  
    greet()
```

Requirements:

- Run the given code to observe the error.
- Apply AI suggestions to correct the syntax.
- Use at least 3 assert test cases to confirm the corrected code works.

Expected Output #1:

- Corrected code with proper syntax and AI explanation.

PROMPT: fix the code

NEW CODE:

```
def greet():  
    print("Hello, AI Debugging Lab!")
```

```
greet()
```

OUTPUT:

Hello, AI Debugging Lab!

Task Description #2 (Incorrect condition in an If Statement)

Task: Supply a function where an if-condition mistakenly uses = instead of ==. Let AI identify and fix the issue.

Bug: Using assignment (=) instead of comparison (==)

```
def check_number(n):  
    if n = 10:  
        return "Ten"  
    else:  
        return "Not Ten"
```

Requirements:

- Ask AI to explain why this causes a bug.
- Correct the code and verify with 3 assert test cases.

Expected Output #2:

- Corrected code using == with explanation and successful test

execution.

Prompt: fix the code

CODE:

```
def check_number(n):  
    if n == 10:  
        return "Ten"  
    else:  
        return "Not Ten"  
  
print(check_number(10))
```

OUTPUTS:

1. Ten
2. Not Ten
3. Not Ten

Task Description #3 (Runtime Error – File Not Found)

Task: Provide code that attempts to open a non-existent file and crashes. Use AI to apply safe error handling.

Bug: Program crashes if file is missing

```
def read_file(filename):  
    with open(filename, 'r') as f:  
        return f.read()  
  
print(read_file('nonexistent.txt'))
```

Requirements:

- Implement a try-except block suggested by AI.

- Add a user-friendly error message.
- Test with at least 3 scenarios: file exists, file missing, invalid path.

Expected Output #3:

- Safe file handling with exception management.

PROMPT:

Fix the code for this outputs file exists, file missing, invalid path.

CODE:

```
def read_file(filename):  
    try:  
        with open(filename, 'r') as f:  
            return f.read()  
  
    except FileNotFoundError:  
        print("Error: The file was not found.")  
        return None  
  
    except OSError:  
        print("Error: Invalid file path or the file cannot be accessed.")  
        return None  
  
# Scenario 1: File exists  
print("---- Scenario 1: File Exists ----")  
content = read_file("existing.txt")  
if content is not None:  
    print("File content:")  
    print(content)  
  
# Scenario 2: File is missing
```

```
print("\n--- Scenario 2: File Missing ---")  
  
read_file("nonexistent.txt")  
  
# Scenario 3: Invalid path  
  
print("\n--- Scenario 3: Invalid Path ---")  
  
read_file("C:/invalid_folder/file.txt")
```

OUTPUTS:

Scenario 1:

File content:

Welcome to the AI Assisted Coding project.

Scenario 2:

Error: The file was not found.

Scenario 3:

Error: Invalid file path or the file cannot be accessed.

Task Description #4 (Calling a Non-Existent Method)

Task: Give a class where a non-existent method is called (e.g., `obj.undefined_method()`). Use AI to debug and fix.

Bug: Calling an undefined method

```
class Car:
```

```
    def start(self):
```

```
        return "Car started"
```

```
my_car = Car()
```

```
print(my_car.drive()) # drive() is not defined
```

Requirements:

- Students must analyze whether to define the missing method

or correct the method call.

- Use 3 assert tests to confirm the corrected class works.

Expected Output #4:

- Corrected class with clear AI explanation.

Prompt:

Bug: Calling an undefined method fix the bug

CODE:

```
class Car:
    def start(self):
        return "Car started"

my_car = Car()
print(my_car.start())
```

OUTPUT:

Car started

Task Description #5 (TypeError – Mixing Strings and Integers in Addition)

Task: Provide code that adds an integer and string ("5" + 2) causing a TypeError. Use AI to resolve the bug.

Bug: TypeError due to mixing string and integer

```
def add_five(value):
    return value + 5

print(add_five("10"))
```

Requirements:

- Ask AI for two solutions: type casting and string concatenation.
- Validate with 3 assert test cases.

Expected Output #5:

- Corrected code that runs successfully for multiple inputs.

PROMPT:

TypeError due to mixing string and integer fix the bug

CODE:

```
def add_five(value):  
    return value + 5  
print(add_five(10))
```

OUTPUT:

15

CODE:

```
def add_five(value):  
    return value + 5  
print(add_five(15))
```

OUTPUT 2:

20

CODE:

```
def add_five(value):  
    return value + 5  
print(add_five(5))
```

OUTPUT 3:

10